

Session : Principale
Régime : RM
Enseignant : Sofiane HACHICHA
Classe(s) : CPI2
Barème : 6 + 14

Date : 05/04/2024
Matière : Programmation Java 2
Durée : 1h
Documents : Non autorisés
Nombre des pages : 4

Devoir Surveillé

Exercice 1

Quelle sera la sortie du programme si on l'exécute en utilisant un JDK 21 ?

```
import java.util.*;
import java.util.stream.*;
public class Exercice1 {
    public static void main(String[] args) {
        List<Integer> l1 = new ArrayList<>(List.of(4, 1, 2, 6, 5));
        System.out.println(Arrays.toString(l1.toArray(new Integer[] {1,2,3,4,5,6,7})));
        SequencedSet<Integer> s1 = new LinkedHashSet<>(l1);
        try {
            l1.add(3);
            System.out.println("l1 :" + l1);
            SequencedSet<Integer> s2 = Collections.unmodifiableSequencedSet(s1);
            s1.addFirst(6);
            System.out.println("s1 :" + s1);
            s2.forEach(System.out::println);
            List<Integer> l2 = l1.subList(1, 4);
            l2.add(1, 2);
            System.out.println("l2 :" + l2);
            l1.addLast(7);
            System.out.println("l1 :" + l1);
            l1.remove(3);
            System.out.println("l1 :" + l1);
        } catch (ConcurrentModificationException e1) {
            System.err.println("Une exception de type ConcurrentModificationException");
        } catch (UnsupportedOperationException e2) {
            System.err.println("Une exception de type UnsupportedOperationException");
        }
        List<Integer> st = s1.stream()
            .filter(c -> c > 3)
            .sorted(Comparator.comparingInt(x -> -x))
            .collect(Collectors.toList());
        System.out.println("st :" + st);
    }
}
```

[4, 1, 2, 6, 5, null, 7]

l1 :[4, 1, 2, 6, 5, 3]

s1 :[6, 4, 1, 2, 5]

6

4

```
1
2
5
12 :[1, 2, 2, 6]
11 :[4, 1, 2, 2, 6, 5, 3, 7]
11 :[4, 1, 2, 6, 5, 3, 7]
st :[6, 5, 4]
```

Exercice 2

Un employé travaillant pour une entreprise vise à superviser la gestion complète de l'inventaire de produits, englobant des aspects tels que le stockage, la classification, le suivi des niveaux de stock et le calcul de la valeur totale du stock par catégorie. Pour mettre en œuvre cette gestion d'inventaire selon les critères définis, nous devons développer deux classes :

- La classe **Produit**, qui représente les différents produits de l'inventaire avec leurs caractéristiques telles que le code, le nom, la quantité en stock et le prix unitaire.
- La classe **Inventaire**, qui gère la liste des produits disponibles, les catégories de produits et la correspondance entre les catégories et les produits.

Une partie du code des classes Produit et Inventaire vous est fournie. A vous de compléter le corps des méthodes signalées.

1. Compléter la classe **Produit** donnée ci-dessous.

```
public class Produit {
    private String codeProduit;    // identifiant unique du produit
    private String nomProduit;     // nom du produit
    private int quantiteStock;     // nombre d'unités disponibles en stock
    private double prixUnitaire;   // prix d'une unité du produit

    public Produit(String codeProduit, String nomProduit, int quantiteStock,
double prixUnitaire) {
        this.codeProduit = codeProduit;
        this.nomProduit = nomProduit;
        this.quantiteStock = quantiteStock;
        this.prixUnitaire = prixUnitaire;
    }

    public String getCodeProduit() {
        return codeProduit;
    }
    public String getNomProduit() {
        return nomProduit;
    }

    public int getQuantiteStock() {
        return quantiteStock;
    }
    public void setQuantiteStock(int quantiteStock) {
        this.quantiteStock = quantiteStock;
    }

    public double getPrixUnitaire() {
        return prixUnitaire;
    }
    public void setPrixUnitaire(double prixUnitaire) {
```

```

        this.prixUnitaire = prixUnitaire;
    }
    // Méthode pour calculer la valeur totale du stock du produit :
    // (quantité en stock * prix unitaire).
    public double calculerValeurStock() {
        // A compléter
        return getQuantiteStock()*getPrixUnitaire();
    }
    // Méthode générique pour comparer le prix unitaire du produit actuel avec
    // celui d'un autre produit de type générique T et de vérifier si le produit
    // actuel est moins cher que l'autre produit.
    public <T extends Produit> boolean estMoinsCherQue(T autreProduit) {
        // A compléter
        return this.prixUnitaire < autreProduit.getPrixUnitaire();
    }
    @Override
    public String toString() {
        return "Produit{" +
            "codeProduit='" + codeProduit + '\'' +
            ", nomProduit='" + nomProduit + '\'' +
            ", quantiteStock=" + quantiteStock +
            ", prixUnitaire=" + prixUnitaire +
            '}';
    }
}

```

2. Compléter la classe **Inventaire** donnée ci-dessous.

```

import java.util.*;
public class Inventaire {
    // Liste des produits disponibles : contient tous les produits gérés par
    // l'inventaire.
    private List<Produit> produitsDisponibles;
    // Ensemble des catégories de produits : représente les différentes
    // catégories de produits disponibles.
    private Set<String> categoriesProduits;
    // Map de catégorie par produits : associe chaque catégorie à une liste
    // de produits correspondants.
    private Map<String, List<Produit>> categorieParProduits;
    // Un constructeur qui initialise la liste des produits disponibles en
    // utilisant une liste chaînée, qui initialise également l'ensemble des
    // catégories tout en préservant l'ordre d'insertion. De plus, il initialise
    // la map categorieParProduits triée par l'ordre naturel des catégories.
    public Inventaire() {
        // A compléter
        produitsDisponibles = new LinkedList<>();
        categoriesProduits = new LinkedHashSet<>();
        categorieParProduits = new TreeMap<>();
    }
    // Méthode pour ajouter un produit à l'inventaire en spécifiant sa catégorie.
    public void ajouterProduit(Produit produit, String categorie){
        // A compléter
        produitsDisponibles.add(produit);
        categoriesProduits.add(categorie);

        List<Produit> p = categorieParProduits.get(categorie);
    }
}

```

```
        if (p == null) {
            p = new ArrayList<>();
            categorieParProduits.put(categorie, p);
        }
        p.add(produit);

        // Ou bien de manière professionnelle et concise.
        // categorieParProduits.computeIfAbsent(categorie, k ->
        // new ArrayList<>()).add(produit);
    }

    // Méthode pour trouver la catégorie d'un produit en recherchant son code
    // dans la map de catégorie par produits.
    private String trouverCategorieProduit(String codeProduit) {
        // A compléter
        for (Map.Entry<String, List<Produit>> entry :
            categorieParProduits.entrySet()) {
            for (Produit produit : entry.getValue()) {
                if (produit.getCodeProduit().equals(codeProduit)) {
                    return entry.getKey();
                }
            }
        }
        return null;
    }

    // Méthode pour supprimer un produit de la liste des produits disponibles et
    // de la map de catégorie par produits.
    public void supprimerProduit(String codeProduit) {
        // A compléter
        Produit produitASupprimer = null;
        for (Produit produit : produitsDisponibles) {
            if (produit.getCodeProduit().equals(codeProduit)) {
                produitASupprimer = produit;
                break;
            }
        }
        if (produitASupprimer != null) {
            produitsDisponibles.remove(produitASupprimer);
            String categorie = trouverCategorieProduit(codeProduit);
            if (categorie != null) {
                categorieParProduits.get(categorie).remove(produitASupprimer);
            }
        }
    }

    // Méthode pour mettre à jour la quantité en stock et le prix unitaire d'un
    // produit existant.
    public void mettreAJourProduit(String codeProduit, int nouvelleQuantite,
        double nouveauPrix) {
        // A compléter
        Produit produitAMettreAJour = null;
        for (Produit produit : produitsDisponibles) {
            if (produit.getCodeProduit().equals(codeProduit)) {
```

```

        produitAMettreAJour = produit;
        break;
    }
}
if (produitAMettreAJour != null) {
    produitAMettreAJour.setQuantiteStock(nouvelleQuantite);
    produitAMettreAJour.setPrixUnitaire(nouveauPrix);
}

}
// Méthode pour afficher la liste des produits disponibles.
public void afficherProduitsDisponibles() {
    // A compléter
    produitsDisponibles.forEach( e -> System.out.println(e));
}
// Méthode pour afficher la valeur totale du stock pour chaque catégorie de
// produits.
public void afficherValeurStockParCategorie() {
    // A compléter
    for (String categorie : categoriesProduits) {
        double valeurTotaleStockCategorie = 0;
        for (Produit produit : categorieParProduits.get(categorie)) {
            valeurTotaleStockCategorie += produit.calculerValeurStock();
        }
        System.out.println("Catégorie : " + categorie + ", Valeur
            totale du stock : " + valeurTotaleStockCategorie);
    }
}
}

```

Indications :

L'interface Collection<E> propose diverses méthodes pour gérer des collections, notamment :

Méthode	Rôle
boolean add(E e)	Ajouter un élément e à la collection.
boolean contains(Object o)	Retourner un booléen précisant si l'objet o est présent dans la collection.
boolean remove(Object o)	Retourner un booléen indiquant si la première occurrence de l'objet o a été supprimée de la collection.
Iterator<E> iterator()	Retourner un Iterator parcourant les éléments de la collection.

L'interface Map<K, V> propose diverses méthodes pour gérer des maps, notamment :

Méthode	Rôle
V put(K key, V value)	Insérer la clé et sa valeur associée spécifiées en tant que paramètres dans la map.
V get(Object key)	Retourner la valeur associée à la clé spécifiée en tant que paramètre.
Set<Map.Entry<K, V>> entrySet()	Retourner un ensemble contenant les paires clé/valeur de la map.
default V computeIfAbsent(K key, Function<? super K, ? extends V> mappingFunction)	Calculer la valeur associée à la clé spécifiée en utilisant une fonction de mappage fournie si la clé n'est pas déjà présente dans la map, et associer cette valeur calculée à la clé dans la map. Si la clé existe déjà dans la map, cette méthode ne modifie pas la map et retourne simplement la valeur associée à cette clé.
V remove(Object key)	Supprimer l'entrée associée à la clé spécifiée et retourner la valeur associée à cette clé avant la suppression, ou null si la clé n'est pas présente dans la map.
Set<K> keySet()	Retourner un ensemble contenant toutes les clés de la map.
Collection<V> values()	Retourner une collection contenant toutes les valeurs associées aux clés présentes dans la map.

L'interface Map<K, V> contient une interface interne Map.Entry<K, V>, qui encapsule une paire clé/valeur et fournit les méthodes getKey() et getValue() pour obtenir la clé et la valeur associées à cette entrée de la map.

BON TRAVAIL